务 过 检查 记唤 进 复
“ 检查 Waker 还没登记”这个窗口里到达时，等待方错过了这次通知，之后就没有人再来唤醒它了。本章大多数改动都对准这个竞争窗口，以及由此派生出的几个具体问题：超时后旧等待项还留在队列里，子进程状态变化和父进程登记 Waker 间 Futex 队 间 样 竞

moss StarryOS RR 调度框架，改动集中在进入阻塞和重新唤醒之间的那段路径。任务在就绪、运行和阻塞之间循环，执行完毕后退出调度流程，整体关系如图所示。

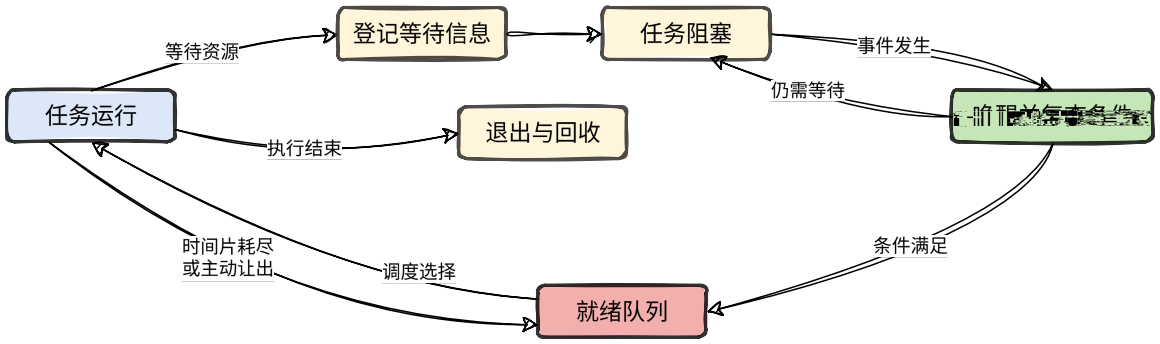


图 3-1 调度、阻塞与唤醒的总体循环

3.1

StarryOS 运 务调 `axtask` 务 态 运 队 `axsched` RR 调度 、 时间 ，多 个 CPU 绪队 Futex WAIT/WAKE 唤 队 robust list 时 进 这 moss StarryOS 进 、

问题集中在 、 错时的状态 ， 。Futex 项 Waker 态 时 记 还 队 —— 达 项 唤 `FUTEX_WAIT_BITSET` 对 时间 对时 时 样 队 进 态变 进 记 Waker 间 样

moss RR 调度框架，改动集中在 个环 ！ 等待 如 登记，通知如 与阻塞状态 ，等待 后如 ， 记 。对 关系如 ：

	StarryOS	moss
务调	务 态 RR 时间 和 CPU 和	调 图
唤	Future Waker 队	调整 PollSet 唤醒过程, 、 复查和等待项
时	进 时	对 对时间 TimerKey 过旧
Futex	WAIT WAKE 队	态 时 、
进		态变 Waker 记 复查

3-1 调度与 、 的

这 再改 , 任务 CPU 、 CPU 过态 、 时, 务检查 记 Waker 时, 、 、 到达 任务准 进入阻塞时, 通知 、 、 , 生。 的 任务状态、 和 、 调度 , 再 这 竞争窗口 通 阻塞唤醒 , 后 时等待、子进程等待和 Futex

3.2

3.2.1

务调 态 调 务 、 务 态 、

```rust
// vendor/axtask/src/task.rs

///
#[repr(u8)]
#[derive(Debug, Clone, Copy, Eq, PartialEq)]
pub enum TaskState {
    ///      CPU
    Running = 1,
    ///
    Ready   = 2,
    ///
    Blocked = 3,
    ///
    Exited  = 4,
}

///
pub struct TaskInner {
    id: TaskId,                         //
```

```
    name: SpinNoIrq<String>,           //
    is_idle: bool,                     //
    is_init: bool,                     //

    entry: Cell<Option<Box<dyn FnOnce()>>>,  //
    state: AtomicU8,                   //

    cpumask: SpinNoIrq<AxCpuMask>,     // CPU
    cpu_id: AtomicU32,                 //                    CPU ID

    #[cfg(feature = "smp")]
    on_cpu: AtomicBool,                //                    CPU

    #[cfg(feature = "preempt")]
    need_resched: AtomicBool,          //
    #[cfg(feature = "preempt")]
    preempt_disable_count: AtomicUsize, //

    interrupted: AtomicBool,           //
    interrupt_waker: AtomicWaker,      //

    exit_code: AtomicI32,              //
    wait_for_exit: AtomicWaker,        //

    kstack: Option<TaskStack>,         //
    ctx: UnsafeCell<TaskContext>,      //

    #[cfg(feature = "task-ext")]
    task_ext: Option<AxTaskExt>,       //

    #[cfg(feature = "tls")]
    tls: TlsArea,                      //
}
```

3-1 务 态 、

`state` 务 态 时 Futex 进 、务 、 CPU 务 `Blocked` `Ready` 调 这 态 。



态 间 、关 图

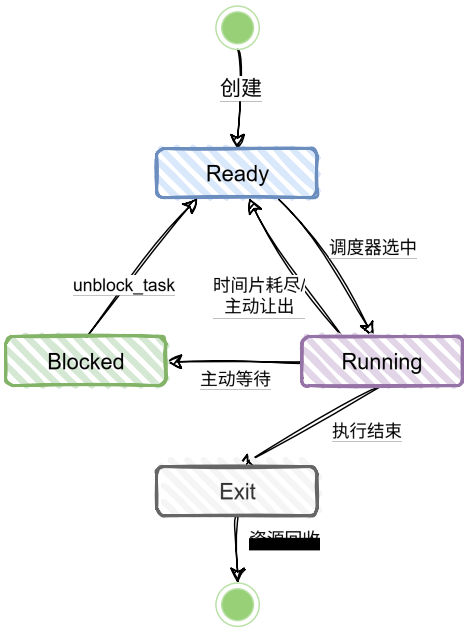


图 3-2 TaskState 态 关

务 Ready 态进 绪队 调 变 Running 动 CPU 务 运 Ready 务 进 Blocked unblock_task 复 Ready 务执 进 Exited 态 moss 时 Futex 进 过这 态 调 态 过 态 运 队 唤 检查 时 达时 唤 Blocked 务 进 绪队

3.2.2

务 RR CPU 环 CPU 绪队 Ready 态 务 调 队 务 务 时间 5 时 时间 务 队 CPU 后 的任务。

时 中 到来 次，scheduler_timer_tick 调 时 务 时间 时 务 执 调

```
// vendor/axtask/src/run_queue.rs

#[cfg(feature = "irq")]
pub fn scheduler_timer_tick(&mut self) {
    let curr = &self.current_task;
    //
    if !curr.is_idle() && self.inner.scheduler.lock().task_tick(curr) {
        #[cfg(feature = "preempt")]
        curr.set_preempt_pending(true);  //
    }
}
```

3-2 调度 时

时 中 → 时间 、 → 、待 _ → → 调

时间 后， 记 _ ，等执行 的 ， 再调度。任务如 在再
改运行队列、 中 ， 有关 时 ， 出， CPU 间 态
， 态 运 队 、
务 运 CPU 进 idle 务 务时运 务
务

3.2.3

务 态 变 时间 ， 调
， 务 态 毕 resched CPU 绪队
务 队 idle 务 时 CPU

```
// vendor/axtask/src/run_queue.rs

fn resched(&mut self) {
    //                                idle
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

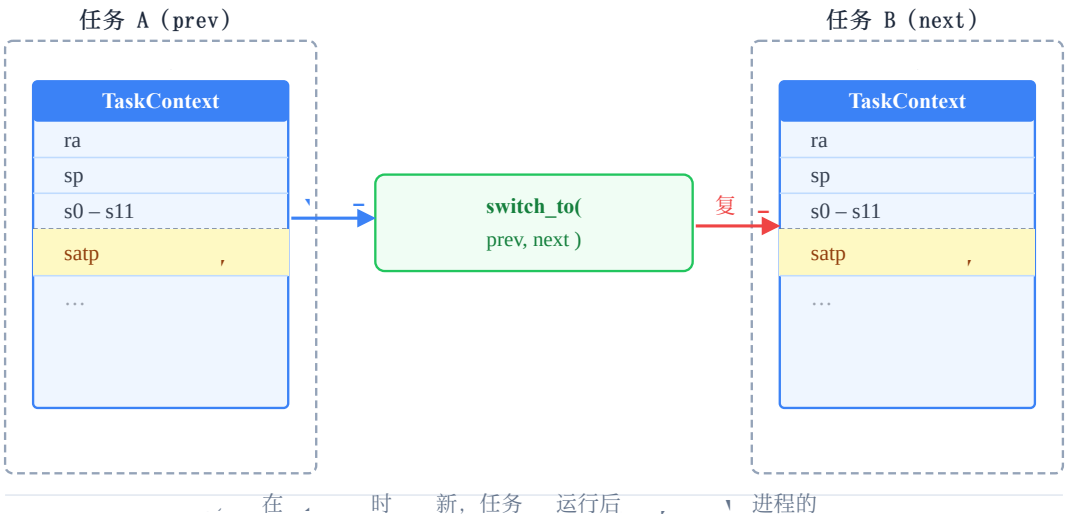
fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    //
    next.set_state(TaskState::Running);

    //                  复
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

3-3 务 ， _

出任务后，switch_to 记 Running ， 务 _ 这 “ _ ” 务
运 务 TaskContext
， 时 CPU 这 务 务 TaskContext 复
务 时 运

图 3-3 务 TaskContext 、 switch_to prev
next 入 中 satp ， 完 的 间就生 了新的 间。



在 时 新，任务 运行后 进程的

图 3-3 TaskContext switch_to

对 任务，moss 务时 进 TaskContext
间 态 UserContext 时 记

3.2.2 这

待 → 达 调 → 任务 → → 复

3.2.4 CPU

务 CPU 和 cpumask CPU 运
CPU 0 CPU 2 时，任务就 进入 CPU 绪队
务 CPU 务 唤 时 调 cpumask
CPU 务 CPU 绪队

```
// vendor/axtask/src/run_queue.rs

///                  CPU
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        //        CPU
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        //                      CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

3-4 CPU

这段 、 CPU , 过 和 的 CPU 运 队
moss 这 图 务

图 对_ CPU moss 这 图 过

CPU 1 务

过 Linux `sched_getaffinity` 查 、 ，通过 `sched_setaffinity`

```
//            每            CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

//
let curr = current().clone();
curr.set_cpumask(cpu_mask);

//     CPU                                CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

3-5 CPU 和 的 、与任务

还 检查 务 CPU CPU 务 ， 运

moss 务 运 队 变 务

运 查

调 `sched_yield` 动 CPU moss 务 绪队 调

务运 这 务 时 绪 务 执

3.3

moss StarryOS Future/Waker 这 timerfd pidfd Futex

唤 `block_on` 复

务 、 态

检查 竞 这

3.3.1

调度 ， 知 任务 ， `Blocked` 务 对 间还

_ moss Future “ 还 ， 检查 ” Waker 记 “

变 时 务”

务 检查 Waker
记 Future “ 时 完 ” 务 调 Waker
务 检查

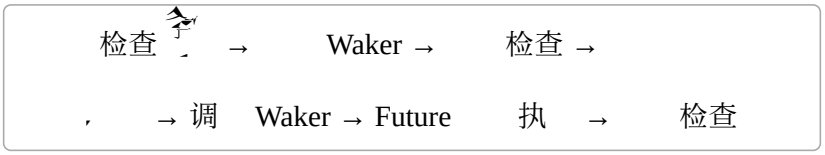


图 3-4 记

关 、 登记 Waker 检查 检查 达
时 还 调 Waker 记 复查 务
达

还 检查 Waker “ 变 ”
务 务 时 时 务 务

3.3.2 Waker PollSet

Waker StarryOS 对 务唤 对 运 队
Waker 态 变 时调 `wake()` moss 这 口， 新
的 timerfd pidfd Futex 务唤

`PollSet` Waker eventfd timerfd pidfd 对 过 记
moss `has_waiters` 时 唤 时 Waker
调 调 `wake()`

```rust
// vendor/axpoll/src/lib.rs

pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    //              wake
    has_waiters: AtomicBool,
}

impl PollSet {
    pub fn register(&self, waker: &Waker) {
        let mut inner = self.inner.lock();
        self.has_waiters.store(true, Ordering::Release);
        inner.register(waker);
    }

    pub fn wake(&self) -> usize {
        if !self.has_waiters.load(Ordering::Acquire) {
            return 0;
        }

        let mut guard = self.inner.lock();
```

```
        if guard.is_empty() {
            self.has_waiters.store(false, Ordering::Release);
            return 0;
        }
        //                    Waker
        let (wakers, len) = guard.take_wakers();
        self.has_waiters.store(false, Ordering::Release);
        drop(guard);  //
        //
        for waker in wakers.iter().take(len) {
            unsafe { waker.assume_init_read() }.wake();
        }
        len
    }
}
```

3-6 PollSet 唤

take_wakers Waker 时 这 记

PollSet “ 检查 ” 、务对 检查 进 检查 态 Futex 检查 记

3.3.3 block_on

Future 过 复 调 、 时 务 block_on、 这 执行方 ：Future Ready 时，系 调 、 Pending 时，、 任务准 阻塞， CPU 、 务 务 _ 、 3.2.3 调

```
// vendor/axtask/src/future/mod.rs

struct AxWaker {
    task: WeakAxTaskRef,           //
    woke: SpinNoIrq<bool>,         //
}

impl Wake for AxWaker {
    fn wake_by_ref(self: &Arc<Self>) {
        if let Some(task) = self.task.upgrade() {
            //
            *self.woke.lock() = true;
            //                  适
            select_run_queue(&task).unblock_task(task, false);
        }
    }
}

pub fn block_on<F: IntoFuture>(f: F) -> F::Output {
    let mut fut = pin!(f.into_future());
```

```
    let curr = current();
    let task = curr.clone();
    let axwaker = AxWaker::new(&task);
    let waker = Waker::from(axwaker.clone());
    let mut cx = Context::from_waker(&waker);

    loop {
        match fut.as_mut().poll(&mut cx) {
            Poll::Pending => {
                let mut rq = current_run_queue::<NoPreemptIrqSave>();
                let woke = axwaker.woke.lock();
                if !*woke {
                    //                                  Blocked
                    rq.blocked_resched(woke);
                } else {
                    //
                    drop(woke);
                    crate::yield_now();
                }
            }
            Poll::Ready(output) => break output,
        }
    }
}
```

3-7 Waker block_on 唤

Future `Pending` 达 ， 、时 Waker `woke` ，再 唤醒任务 阻塞路径 在改变任务状态 检查这个 记。通知 到时，任务 再进入 `Blocked` CPU 时 `unblock_task` 务 绪队 `woke` 记 “ ”、 Future 态 务 态 间

图 3-5 对 了这 路径的时 。

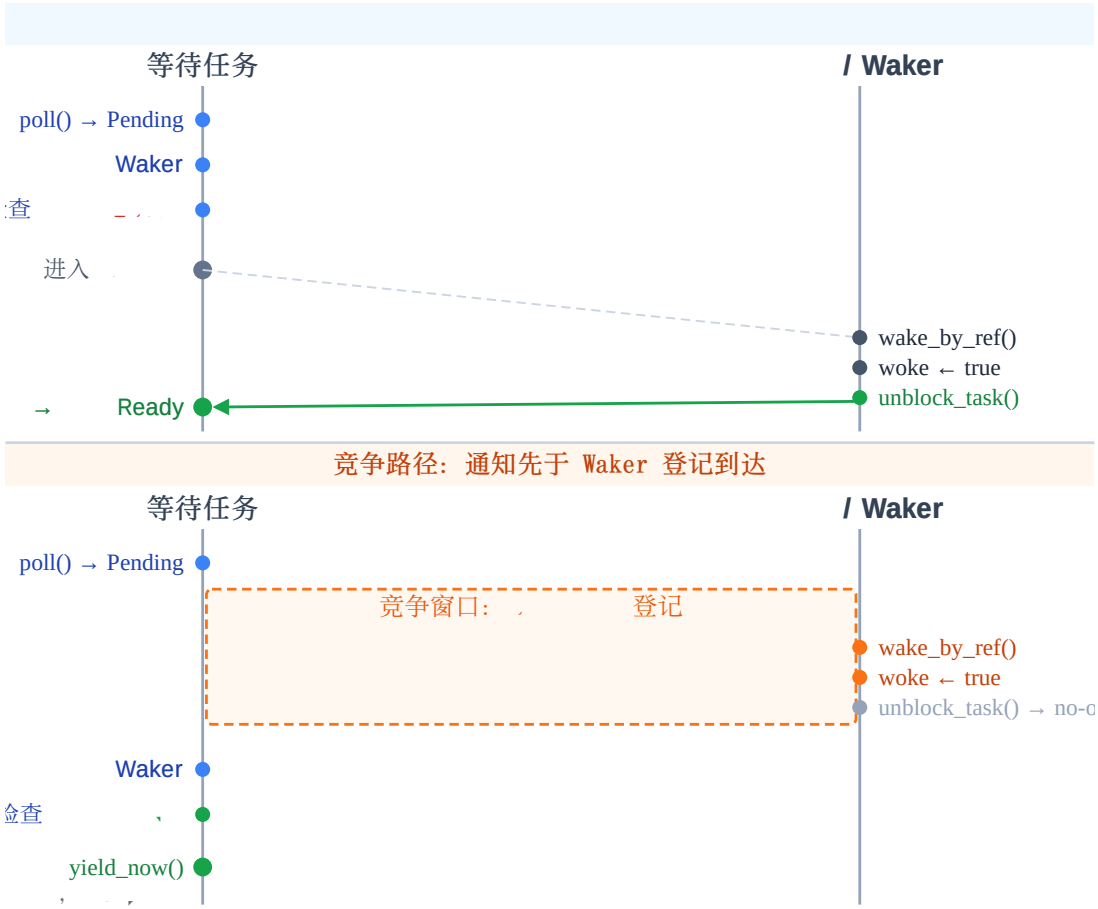


图 3-5 block_on woke 记 竞争路径的时 对

、 interruptible 时检查 Future 务

态 、 、 调 EINTR 时间

时 、 还 记 Waker

唤 唤 PollSet Waker Futex 、 等待项

，具体过程。在 3.5 。

3.4

3.3 了“检查 、登记 Waker 复查”

。 这 时 关 时间 达 进 关

态 、 务 唤 、 务 复 检

查

3.4.1

时 调 时间 时间 调时

进 “ ” 时时 对。 时间 时

时间 数 对时 和 对时间。 对时 、 到 时间 ， 对时间本 就

时间 如 时间 到达，等待 、 。 后，后 、 ， 、 “ 时间

达 时间” 样 FUTEX_WAIT_BITSET 时 、

nanosleep 调时 对 clock_nanosleep 还 时 ， 通过 TIMER_ABSTIME 对时间 间 moss 、 EINTR 时间

3.4.2

StarryOS 时 时间 ， 时 队 时间 moss BTreeMap 时 时间 deadline seq 的 TimerKey 时 务 时 态 ， _

```rust
struct TimerKey {
    //
    deadline: Duration,
    //
    seq: u64,
}

impl PartialOrd for TimerKey {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.cmp(other))
    }
}

impl Ord for TimerKey {
    fn cmp(&self, other: &Self) -> Ordering {
        //
        self.deadline.cmp(&other.deadline)
            .then_with(|| self.seq.cmp(&other.seq))
    }
}

struct AlarmRuntime {
    //
    next_seq: u64,
    //
    timers: BTreeMap<TimerKey, Entry>,
}
```

3-8 时 队 关 、

时 时 seq 时 时 务 还 时 ， 时 时 还 务 时间

还 时间 对 TimeManager 、 态 态运 时间 时间 时 ITIMER_REAL 过时间， SIGALRM 时间 时 ITIMER_VIRTUAL

态时间 时 ITIMER_PROF 时 态 态时间 时
时

3.4.3

进 态 、 过 务
唤 进 时 态时 记 运 进 态变
进 唤
进 调 wait 时 子进程，再检查 、 有 、 状态。如 时没有 ，它就 Waker 记 child_exit_event 进 进 态 变 时 ，这
进 复运 态

```
//
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        //                    Waker
        child_exit_event.register(cx.waker());
        //      复
        match check_children().transpose() {
            Some(result) => Poll::Ready(result),
            None => Poll::Pending,
        }
    }
}
```

3-9 进 进 态

进 环查 进 态变 ，Waker 进 复 执 check_children 项 、 记 复查 3.3.1 “ 检查 Waker 记” 间， 进 态变

3.5 Futex

3.5.1

Futex 竞 态 过 变 ， ，时 进 竞 调
进入 后，moss FutexKey 队 进 Futex
还 、 进 、
Futex

```
// kernel/src/task/futex.rs

/// Futex                    Futex
pub enum FutexKey {
    ///          Futex
    Private {
```

```
        address: usize,  //
    },

    ///                 Futex
    Shared {
        offset: usize,   // Futex
        region: Result<Weak<SharedPages>, Weak<()>>,  //
    },
}

impl FutexKey {
    ///                       FutexKey
    pub fn new(aspace: &AddrSpace, address: usize) -> Self {
        if let Some(area) = aspace.find_area(VirtAddr::from_usize(address)) {
            match area.backend() {
                Backend::Shared(backend) => {
                    return Self::Shared {
                        offset: address - area.start().as_usize(),
                        region: Ok(Arc::downgrade(backend.pages())),
                    };
                }
                Backend::File(file) => {
                    return Self::Shared {
                        offset: address - area.start().as_usize(),
                        region: Err(file.futex_handle()),
                    };
                }
                _ => {}
            }
        }
        Self::Private { address }
    }
}
```

3-10 FutexKey 、

Futex 项 队 owner_dead 态 唤 时 moss 项

```
// kernel/src/task/futex.rs

/// Futex
#[derive(Default)]
pub struct WaitQueue {
    // 每个等待 是一个三元 :
    // (       ,          ,           )
    queue: SpinNoIrq<VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>>,
}
```

```rust
/// Futex
pub struct FutexEntry {
    ///
    pub wq: WaitQueue,
    ///      robust list        Futex
    pub owner_dead: AtomicBool,
}

impl WaitQueue {
    ///
    pub fn wait_if(
        &self,
        bitset: u32,
        timeout_dur: Option<Duration>,
        condition: impl FnOnce() -> bool,
    ) -> AxResult<bool> {
        let notified = Arc::new(AtomicBool::new(false));
        let poll = Arc::new(PollSet::new());

        {
            let mut queue = self.queue.lock();
            if !condition() {
                return Ok(false);  //
            }
            //
            queue.push_back((notified.clone(), bitset, poll.clone()));
        }

        //
        let result = block_on(interruptible(timeout(
            timeout_dur,
            poll_fn(|cx| {
                if notified.load(Ordering::Acquire) {
                    return Poll::Ready(());
                }
                poll.register(cx.waker());
                if notified.load(Ordering::Acquire) {
                    Poll::Ready(())
                } else {
                    Poll::Pending
                }
            }),
        )));

        match result {
```

```
            Ok(Ok(())) => Ok(true),  //
            Ok(Err(_)) => {
                //
                let mut queue = self.queue.lock();
                queue.retain(|(waiter, ..)| !Arc::ptr_eq(waiter, &notified));
                Err(AxError::TimedOut)
            }
            Err(_) => {
                //
                let mut queue = self.queue.lock();
                queue.retain(|(waiter, ..)| !Arc::ptr_eq(waiter, &notified));
                Err(AxError::Interrupted)
            }
        }
    }

    ///         count              bitset    mask
    pub fn wake(&self, count: usize, mask: u32) -> usize {
        let mut woke = 0;
        self.queue.lock().retain(|(waiter, bitset, poll)| {
            if woke >= count || (bitset & mask) == 0 {
                true  //
            } else {
                waiter.store(true, Ordering::Release);
                poll.wake();
                woke += 1;
                false  //
            }
        });
        woke
    }
}
```

3-11 Futex 队 、

AtomicBool notified 记 项 唤 u32 bitset PollSet 务 唤 、务 态 唤 动 notified 复 PollSet 任务重新 与调度。

图 3-6 FutexKey WaitQueue 项 ——notified 记 通知 ，bitset 过 唤醒 ，PollSet 唤

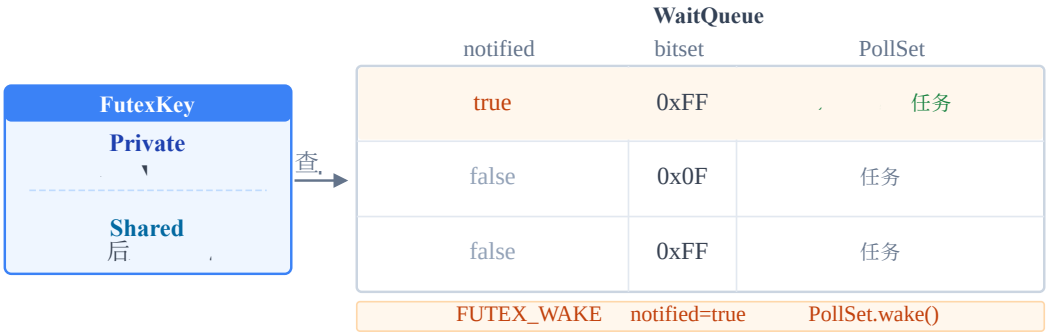


图 3-6 FutexKey WaitQueue 项 关

3.5.2 WAIT WAKE

时 调 检查 `EAGAIN` 队时 队 检查 队 进 间 变时 调 时 项 进 时 Future `FUTEX_WAKE` 队 `notified` ，再调 PollSet 唤 项 时 队 wake

3.3.3 这 Futex 项 队 竞 `notified` `FUTEX_WAKE` 记 PollSet 复查 记 Futex 项 唤 时 队 态

3.5.3

唤 队 `notified` 态 `FUTEX_WAIT_BITSET` 等待 ，`FUTEX_WAKE_BITSET` 唤 bitset mask 务 关

`FUTEX_REQUEUE` 唤 务 项 动 Futex 队列。`FUTEX_CMP_REQUEUE` 还 时 `EAGAIN` 动过 `notified` bitset PollSet 记 Waker

```
// kernel/src/task/futex.rs

impl WaitQueue {
    ///        count
    pub fn requeue(&self, mut count: usize, target: &WaitQueue) -> usize {
        let tasks: Vec<_> = {
            let mut wq = self.queue.lock();
            count = count.min(wq.len());
            wq.drain(..count).collect()  //
        };
        if !tasks.is_empty() {
            let mut wq = target.queue.lock();
            wq.extend(tasks);  //                          notified  bitset
PollSet
```

```
        }
        count
    }
}
```

3-12 Futex 队

这 ， pthread 变 时 时 唤

对_ 队 图 3-7 、 队

项 notified bitset PollSet Futex 队列。

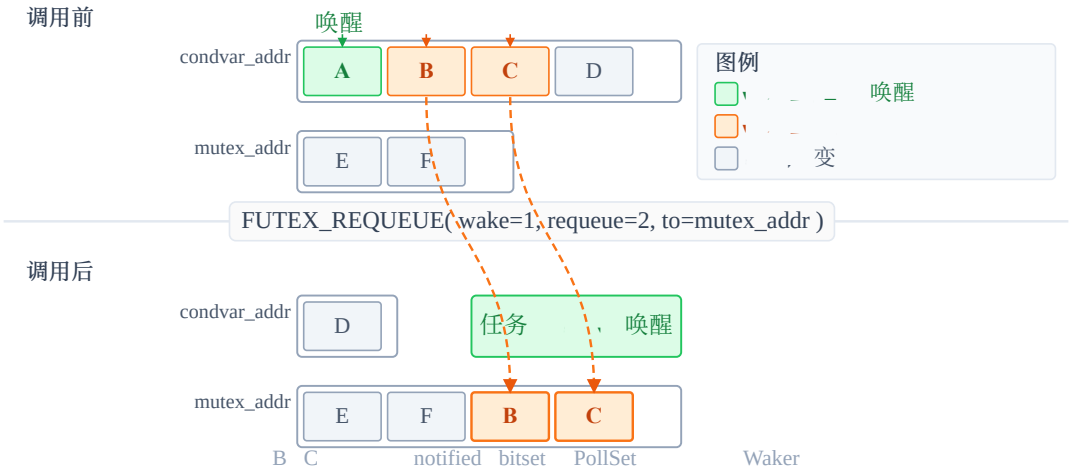


图 3-7 FUTEX_REQUEUE 唤 Futex 队列

3.5.4

Futex 时 moss

interruptible(timeout(...)) 这 路径。超时 ETIMEDOUT EINTR

、 notified 对 项

FUTEX_WAIT FUTEX_WAIT_BITSET 时间 、 moss Linux

： 对时 ，后 对 时间， FUTEX_CLOCK_REALTIME 时时

， 调时 。

态 “ 达” 问题 复查 记

项 、 复 队 关 复

LoongArch64 futex_wait pthread cancellation 、 问题

3.5.5 robust list

程 有 时如 退出， 程 、 等 到 ， 、 。 对这

_ ， moss 时 Futex 态 过 set_robust_list robust list

， ， 记 程 、 有的 Futex

程退出时，moss futex_offset 对_ Futex owner_dead ， 唤醒 个等

待 。 唤醒 程 到 EOWNERDEAD 态 检查 复 、

ROBUST_LIST_LIMIT 、 环

程退出还 ， clear_child_tid 唤 这 对_ Futex 这 路径

pthread_join

3.6

StarryOS RR 调度、任务状态 、 、 、 和 、 等待，moss

调 调 进 调 记 态

